GUIDELINES BASED MCA MAJOR PROJECT REPORT

For

Major Project

(23ONMCR-753)

Scalable Backend Development for High-Performance E-Commerce Website Platform

Submitted in partial fulfillment of the requirements for the programme

Master of Computer Applications

Fourth Semester

Submitted By: _____

Enrollment / Roll No.: _____

Project Guide: _____

CENTRE FOR DISTANCE & ONLINE EDUCATION

CHANDIGARH UNIVERSITY

Certificate

This is to certify that the major project report entitled ’Scalable Backend Development for High-Performance E-Commerce Website Platform’ has been prepared and submitted by _____ in partial fulfillment of the requirements for the Master of Computer Applications programme of Chandigarh University.

The work presented in this report has been carried out as an individual major project. The report follows the official Major Project guidelines provided for course code 23ONMCR-753. The project work is based on the submitted repository documents, source code, database design files, and implementation notes.

To the best of my knowledge, the project report gives a clear and honest description of the system design and the current implemented modules.

Project Guide Signature: _____

Name of Guide: _____

Date: _____

Place: _____

Declaration

I hereby declare that the major project report entitled ’Scalable Backend Development for High-Performance E-Commerce Website Platform’ is based on my project documents and source code. The project has been prepared as an individual academic submission for the MCA Major Project course.

The information written in this report has been taken from the uploaded project guideline PDF and the project files present in the repository. Where personal details such as student name, enrollment number, and guide name were not available, blank fields have been kept so that they can be filled with correct information.

I also declare that no unsupported claim has been added as a completed implementation. Modules that are documented as design or future work are described in the same way in this report.

Student Signature: _____

Student Name: _____

Date: _____

Acknowledgement

I would like to express my sincere thanks to Chandigarh University and the Centre for Distance & Online Education for giving me the opportunity to complete this MCA Major Project.

I am thankful to my project guide, faculty members, and evaluators for their support and guidance. Their direction helped me understand how a major project should be planned, documented, implemented, and tested.

I also thank everyone who helped me directly or indirectly during the preparation of this project. This work helped me learn practical software engineering concepts such as microservices, authentication, database design, API design, testing, monitoring, and scalable deployment.

Finally, I thank my family and friends for their encouragement during the project work.

Abstract

The project titled ’Scalable Backend Development for High-Performance E-Commerce Website Platform’ is a scalable e-commerce platform designed for high traffic online shopping use cases. The system is planned as a microservices-based platform where each major business area, such as authentication, user profile, product catalog, cart, order, payment, search, CMS, session analytics, notification, and superadmin, is handled by a separate service.

The project uses Go for backend service development and React with TypeScript for frontend dashboard development. MySQL is used for strongly structured and transactional data such as authentication, orders, payments, CMS, and admin records. MongoDB is used for flexible and high-volume data such as product documents, cart documents, wishlist records, recommendation data, session events, and notification payloads. Redis is used for fast temporary data such as active sessions, OTP rate limits, carts, and cache. Typesense is planned for search, while Kafka or RabbitMQ is planned for event-driven communication.

The current repository contains full architecture documentation, database schema design, API references, task implementation guides, a Go-based Auth Service implementation, and a React TypeScript Session Analytics Dashboard. Other services are described as part of the complete platform design and future implementation plan. The report explains the objectives, requirements, SDLC, system design, implementation details, testing approach, applications, conclusion, and bibliography in simple academic language.

The main goal of this project is to show how a modern e-commerce system can be designed for modular development, security, observability, and future scalability.

Table of Contents

Introduction

1.1 Project Overview

E-commerce platforms are used by buyers, sellers, administrators, and support teams at the same time. A small online store can be built as a single application, but a high-performance platform needs a design that can grow safely. This project presents a scalable e-commerce platform where different parts of the system are divided into separate services.

The project topic was inferred from the repository README and documents as ’Scalable Backend Development for High-Performance E-Commerce Website Platform’. The system is not only a shopping website idea. It also includes seller tools, superadmin tools, session analytics, payment flow, database design, DevOps planning, logging, monitoring, and security.

1.2 Source Documents Studied

- Official MCA Major Project guideline PDF named Project Work-Assignment 1.pdf.
- README.md describing the project topic.
- Architecture, microservice, database, security, payment, session, frontend, DevOps, monitoring, and developer guide documents in the docs folder.
- Relational and MongoDB database design documents in the database folder.
- Master API reference in api/master-api.json.
- Auth Service source code, migrations, use cases, repositories, and tests.
- Session Analytics Dashboard React TypeScript source code and tests.
- Task-wise implementation notes under TaskImplementation.

1.3 Objectives

- To design a high-performance e-commerce platform using a microservices architecture.

- To separate major business features into independent modules with clear ownership.
- To design secure authentication, authorization, OTP, token, and role management.
- To design catalog, cart, order, payment, seller CMS, superadmin, search, and analytics workflows.
- To use proper database choices for different data types instead of using one database for all data.
- To support future scalability through Redis caching, message queues, Kubernetes, and observability tools.
- To implement selected working modules in Go and React TypeScript.
- To prepare proper academic documentation following the MCA Major Project guidelines.

1.4 Problem Statement

A normal e-commerce application can become difficult to manage when the number of users, products, orders, and sellers increases. If all features are kept inside one large codebase and one shared database, changes in one area can affect other areas. The system can become slow, risky to deploy, and difficult to scale.

The problem addressed by this project is to design an e-commerce platform that can handle different business areas independently. The platform should support secure login, product browsing, cart, checkout, payments, seller management, admin control, session analytics, monitoring, and future growth.

1.5 Scope of the Project

The scope of the project includes both system design and selected practical implementation. The documents define the full platform architecture. The current code implementation mainly includes the Auth Service and the Session Analytics Dashboard. This report describes planned modules and implemented modules separately so that the report remains honest and clear.

Area	Included Scope
Architecture	API Gateway, microservices, databases, Redis, queue, Kubernetes, monitoring.
Backend implementation	Go Auth Service with password, OTP, token, role, session-link, repositories, and migrations.
Frontend implementation	React TypeScript Session Analytics Dashboard shell with filters and metric cards.
Database design	MySQL tables, MongoDB collections, Typesense schema, indexes, and ownership rules.
Security	JWT, refresh token rotation, OTP hashing, RBAC, rate limiting, audit logging, privacy controls.
Deployment planning	Docker, Kubernetes, CI/CD, external services, logging, monitoring, and scaling plan.

1.6 Software Requirement Specification Summary

The Software Requirement Specification describes what the proposed system should provide. In this project, requirements are divided into functional requirements and non-functional requirements. Functional requirements explain the features. Non-functional requirements explain the quality of the system.

1.6.1 Functional Requirements

Requirement	Simple Description
User registration and login	The system should allow users to create account, login, logout, and manage sessions.
OTP verification	The system should support email or phone OTP for verification and password reset.
Role-based access	The system should allow access based on roles such as buyer, seller, admin, and superadmin.
Product catalog	The system should support product, variant, category, brand, price, image, and inventory data.
Search and filters	The system should allow users to search products and filter by brand, category, price, stock, and rating.
Cart and wishlist	The system should allow add, update, remove, cart merge, coupon preview, and wishlist actions.
Checkout and order	The system should create order from cart using idempotency and fresh stock validation.
Payment and refund	The system should create payment intent, verify webhook, update status, and support refund flow.

Requirement	Simple Description
Seller dashboard	The system should allow sellers to manage products, orders, coupons, staff, and analytics.
Superadmin panel	The system should allow admins to manage users, sellers, payments, settings, search, and audit logs.
Session analytics	The system should track sessions, events, journeys, funnels, heatmaps, and live metrics.
Notification	The system should send OTP, order status, payment, refund, and price-drop notifications.

1.6.2 Non-Functional Requirements

Requirement	Expected Quality
Security	Passwords, OTPs, tokens, card data, and secrets must be protected and not logged.
Scalability	Services should scale independently based on traffic and queue load.
Performance	Search, cart, auth, and checkout should respond quickly under normal load.
Reliability	Important operations should use idempotency, retries, and durable events.
Maintainability	Code should be split into domain, usecase, repository, transport, and config layers.
Observability	Services should produce structured logs, metrics, traces, health checks, and alerts.
Privacy	Session analytics should mask PII, hash sensitive values, and support deletion/retention rules.
Compatibility	Frontend should support modern browsers and responsive dashboard layouts.
Extensibility	New services and modules should be added without changing unrelated services.
Deployment readiness	Docker, Kubernetes, config validation, and CI/CD should be supported.

1.7 Assumptions

- The project name was taken from README.md because the user prompt contained a placeholder project name.
- Student name, enrollment number, guide name, and institute signature details were not provided, so blank fields are kept.

- Official logo image files were not provided. Therefore, the report uses a clean text header instead of logo images.
- The full e-commerce platform is described as the target design. The current repository implementation is limited mainly to Auth Service and Session Analytics Dashboard.
- The report uses simple English and avoids complex wording as requested.

1.8 Hardware Requirements

The official guideline gives a minimum hardware direction such as Pentium-IV processor or above, 2 GB RAM, 40 GB hard disk, graphics card, and other required interfaces. For this modern project, the practical development and testing setup can use the following hardware.

Component	Recommended Requirement	Reason
Processor	Intel i5/Ryzen 5 or above	Runs backend services, frontend dev server, and local tools smoothly.
RAM	8 GB minimum, 16 GB recommended	Useful for Docker, database containers, tests, and browser tools.
Storage	40 GB minimum, SSD recommended	Stores source code, databases, packages, logs, and generated builds.
Network	Internet connection	Required for package setup, external APIs, and deployment work.
Display	Standard monitor	Required for dashboard and report verification.

1.9 Software Requirements

Category	Software / Tool	Purpose
Operating System	Linux / Windows / macOS	Development and testing environment.
Backend	Go 1.24+	Backend microservice implementation.
Frontend	React, TypeScript, Vite, Tailwind CSS	Session Analytics Dashboard and future web apps.
Database	MySQL 8+, MongoDB	Structured transactional data and flexible document/event data.
Cache / Session	Redis	Rate limits, active sessions, OTP counters, cart cache.
Search	Typesense	Fast product search and facets.

Category	Software / Tool	Purpose
Messaging	Kafka or RabbitMQ	Asynchronous events and background processing.
Deployment	Docker, Kubernetes	Containerization and scalable deployment.
Monitoring	Prometheus, Grafana, Loki, OpenTelemetry	Metrics, logs, and traces.
Testing	Go test, Vitest, Testing Library, MSW	Backend and frontend test coverage.

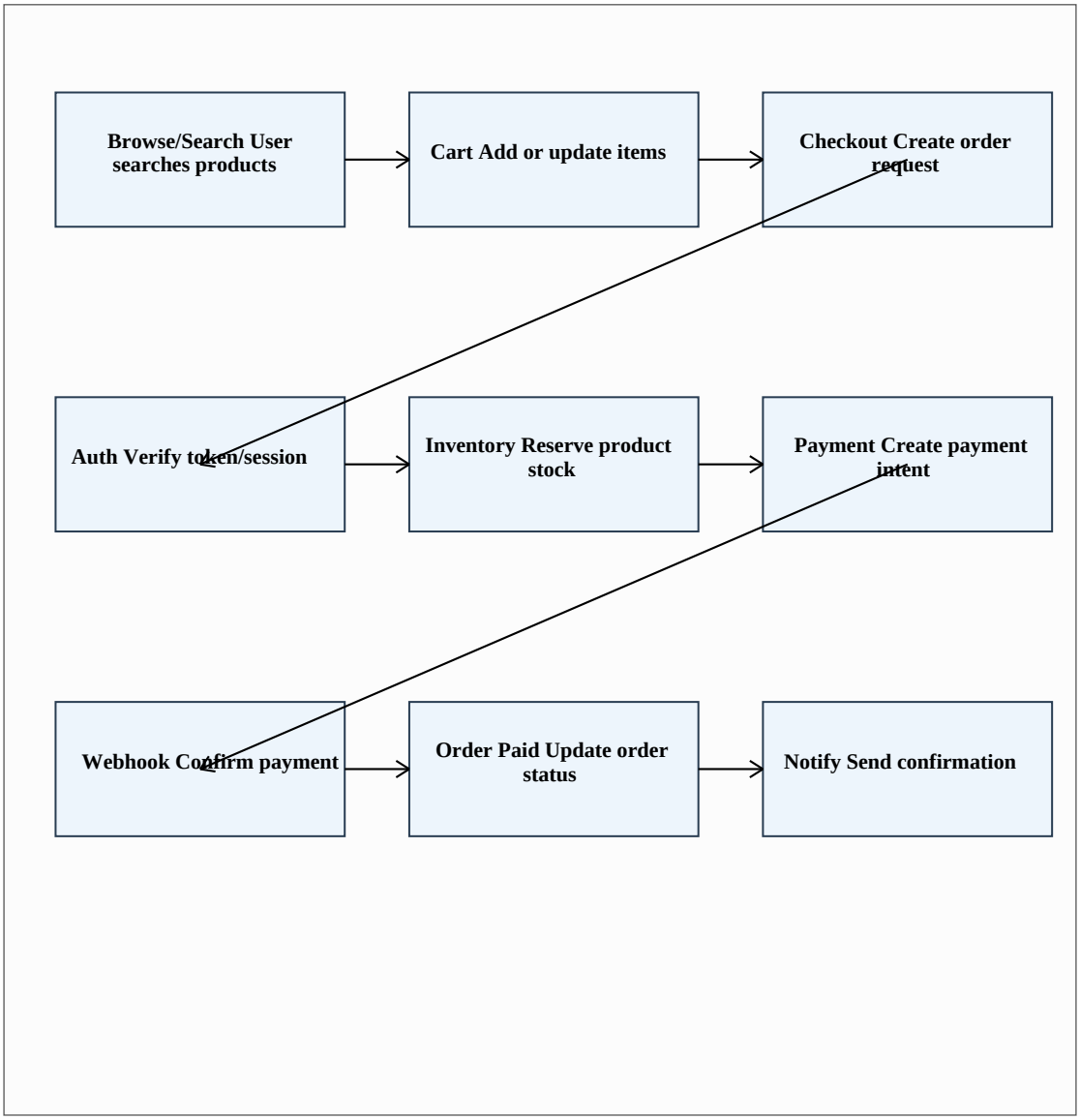
1.10 Broad Areas of Application

- Database Management Systems
- Computer Communication and Networking
- Mobile and Web Application Development
- Software Engineering
- Cloud Deployment and DevOps
- Security and Authentication
- Analytics and Monitoring

SDLC of the Project

The project follows an iterative SDLC approach. In this method, the system is not built in one large step. First, the requirements and architecture are defined. Then modules are designed and implemented step by step. Testing and review are done after each important module.

Figure 1: SDLC Flow Used in the Project



Explanation: The diagram shows a simplified implementation journey from user browsing to notification. The same step-by-step approach was used for project planning and module development.

2.1 Identification Phase

In the identification phase, the main project idea was selected: a scalable backend development project for a high-performance e-commerce platform. The tools and technologies were identified from the project documents and code. Go was selected for backend services because it is fast and suitable for network services. React TypeScript was selected for frontend dashboards. MySQL, MongoDB, Redis, Typesense, and Kafka/RabbitMQ were selected according to the type of data and workload.

2.2 Requirement Analysis Phase

In this phase, functional and non-functional requirements were identified. Functional requirements explain what the system should do. Non-functional requirements explain how well the system should work, such as security, performance, reliability, and scalability.

Requirement Type	Requirement
Functional	User should register, login, refresh token, logout, and verify OTP.
Functional	Buyer should browse products, search, manage cart, and complete checkout.
Functional	Seller should manage products, inventory, orders, coupons, and analytics.
Functional	Admin should manage users, sellers, payments, search, sessions, and audit logs.
Functional	Session Analytics Dashboard should show live metrics using filters.
Non-functional	System should be secure with JWT, refresh rotation, OTP hashing, RBAC, and audit logs.
Non-functional	System should scale using microservices, caching, message queue, and Kubernetes.
Non-functional	System should provide logs, metrics, traces, health checks, and alerts.

2.3 Analysis Phase

The system was analyzed as a set of independent services. Each service owns its own data. For example, Auth Service owns credentials and tokens, Product Service owns product catalog data, Order Service owns orders, and Payment Service owns payment records. Other services should not directly read or write a service's database. They should communicate through APIs or events.

2.4 Design Phase

In the design phase, the system architecture, data flow, database design, module boundaries, security controls, payment flow, session analytics, and deployment plan were prepared. The design uses API Gateway for public access, gRPC for internal service communication, and message queues for asynchronous events.

2.5 Implementation Phase

The implementation available in the repository focuses on two practical areas. The first is the Go Auth Service, which includes HTTP handlers, use cases, password hashing, token issue and verification, OTP flow, RBAC, MySQL repositories, Redis OTP rate repository, outbox events, and migrations. The second is the React TypeScript Session Analytics Dashboard, which includes layout, filters, metric cards, API mapping, validation, error states, and unit/component tests.

2.6 Testing Phase

Testing is planned at different levels. Backend testing includes unit tests for use cases, token security tests, role security tests, password tests, OTP tests, and HTTP security tests. Frontend testing includes helper tests, API mapping tests, component tests, and dashboard page tests. Integration and end-to-end tests are planned for full workflows such as signup, login, search, cart, checkout, payment success, seller product creation, and admin refund review.

2.7 Deployment and Maintenance Phase

Deployment is planned using Docker and Kubernetes. Services run as containers. Kubernetes manages service discovery, scaling, health checks, and rolling deployments. Monitoring uses Prometheus, Grafana, Loki, and OpenTelemetry. Maintenance includes schema migrations, security updates, monitoring alerts, backup planning, and future feature development.

2.8 SDLC Mapping Table

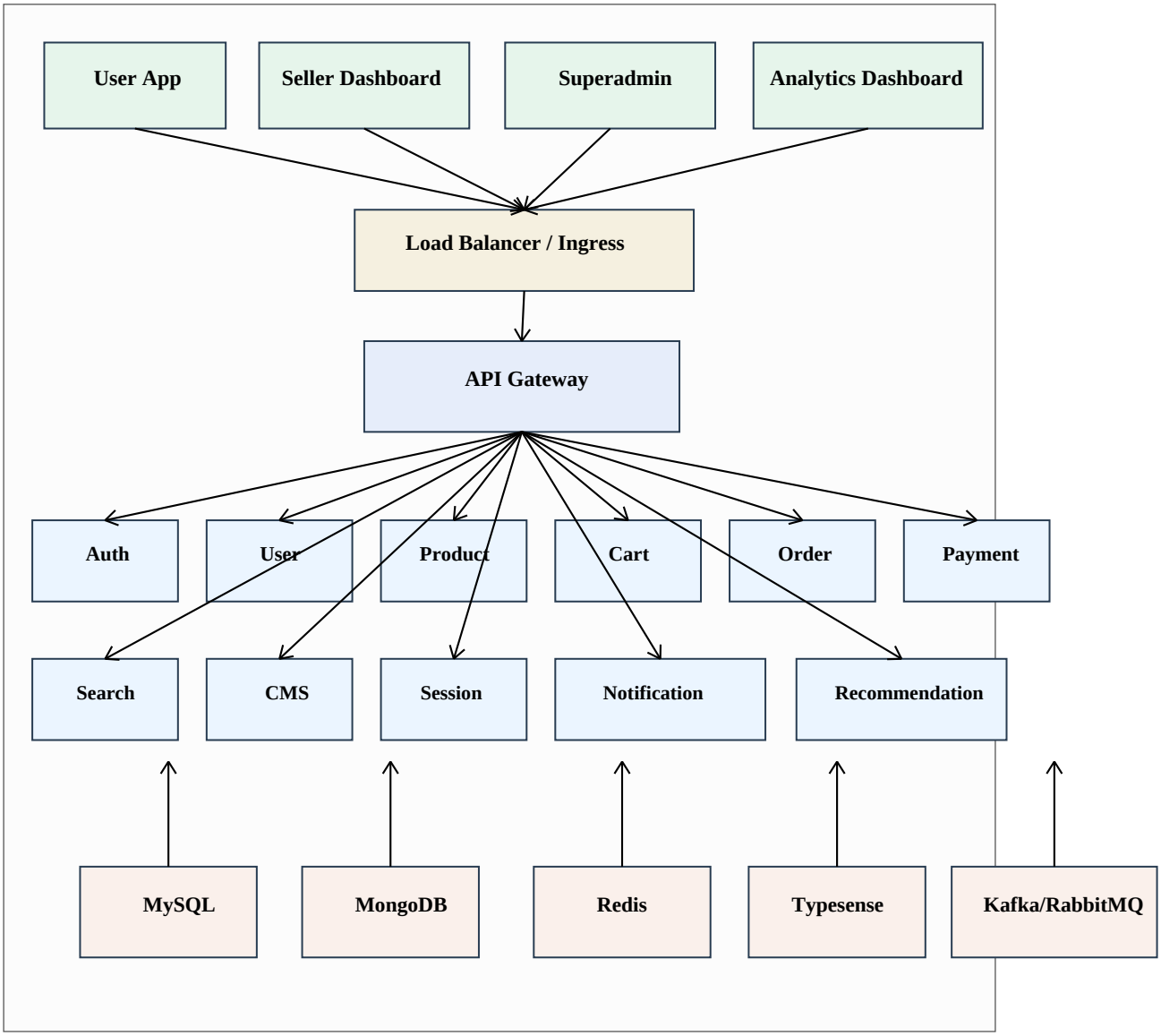
SDLC Phase	Project Work
Identification	Selected scalable e-commerce platform and identified tools.
Requirement Analysis	Prepared functional and non-functional requirements.
System Design	Prepared microservice architecture, database design, API design, and security design.
Implementation	Implemented Auth Service and Session Analytics Dashboard modules.
Testing	Added backend and frontend tests for implemented modules.
Deployment Planning	Documented Docker, Kubernetes, CI/CD, monitoring, and scaling approach.
Maintenance	Prepared future enhancement and production readiness plan.

Design

3.1 Overall System Architecture

The platform is designed as a microservices system. Public requests from web apps go through the Load Balancer, Kubernetes Ingress, and API Gateway. The API Gateway validates requests, checks authentication where required, adds request IDs, performs rate limiting, and routes requests to the correct backend service.

Figure 2: System Architecture

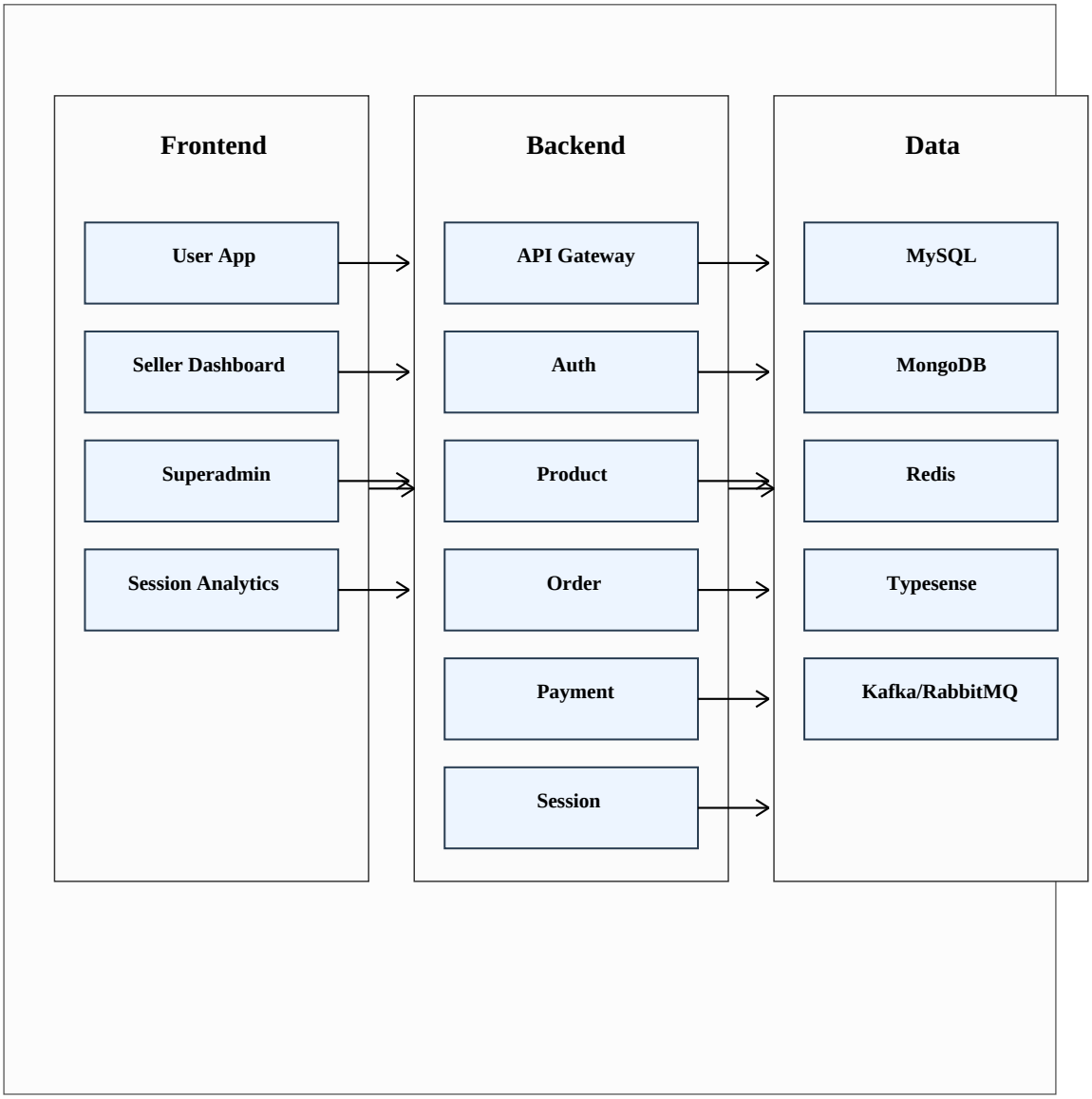


Explanation: The diagram shows the major frontend apps, gateway, backend services, databases, cache, search engine, and message queue. The API Gateway is the main entry point for public traffic.

3.2 Main Modules

Module	Main Responsibility
API Gateway	Public REST entry point, routing, validation, auth checks, rate limiting, and error mapping.
Auth Service	Login, password validation, JWT, refresh tokens, OTP, RBAC, and logout.
User Service	User profile, addresses, seller profile, KYC metadata, and user status.
Product Service	Catalog, variants, attributes, inventory, categories, and product publishing.
Cart Service	User and guest carts, item quantity, coupon preview, and cart merge.
Wishlist Service	Wishlist items, move to cart, price drop and availability tracking.
Order Service	Cart to order conversion, order lifecycle, fulfillment, and cancellation.
Payment Service	Payment intents, provider webhooks, refunds, reconciliation, and audit.
Search Service	Typesense product search, autocomplete, facets, synonyms, and reindexing.
CMS Service	Seller product workflow, coupons, campaigns, seller settings, and analytics.
Session Service	Session events, user journey, live metrics, funnels, heatmaps, and privacy.
Notification Service	Email, SMS, push notifications, templates, delivery logs, and retries.
Superadmin Service	User, seller, order, payment, search, settings, sessions, and audit controls.

Figure 3: Module Diagram

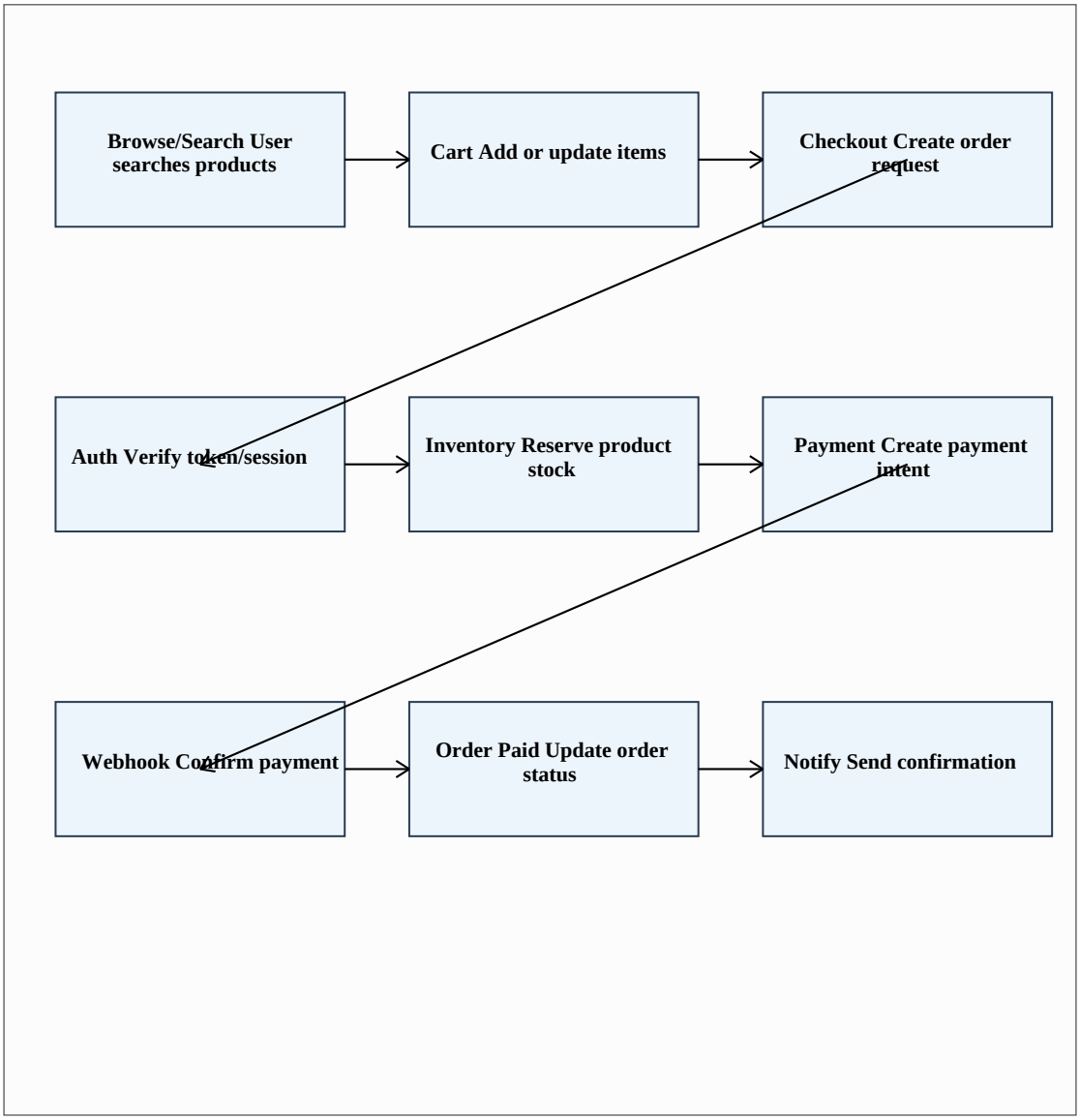


Explanation: The module diagram groups the system into frontend apps, backend services, and data/infra components. This makes the ownership of each part easy to understand.

3.3 Workflow Diagram

A typical e-commerce workflow starts when a user searches or browses a product. The user adds an item to the cart and starts checkout. The system verifies the user session, creates an order, reserves inventory, creates a payment intent, waits for the provider webhook, marks the order as paid, commits inventory, and sends notification.

Figure 4: Checkout and Payment Workflow

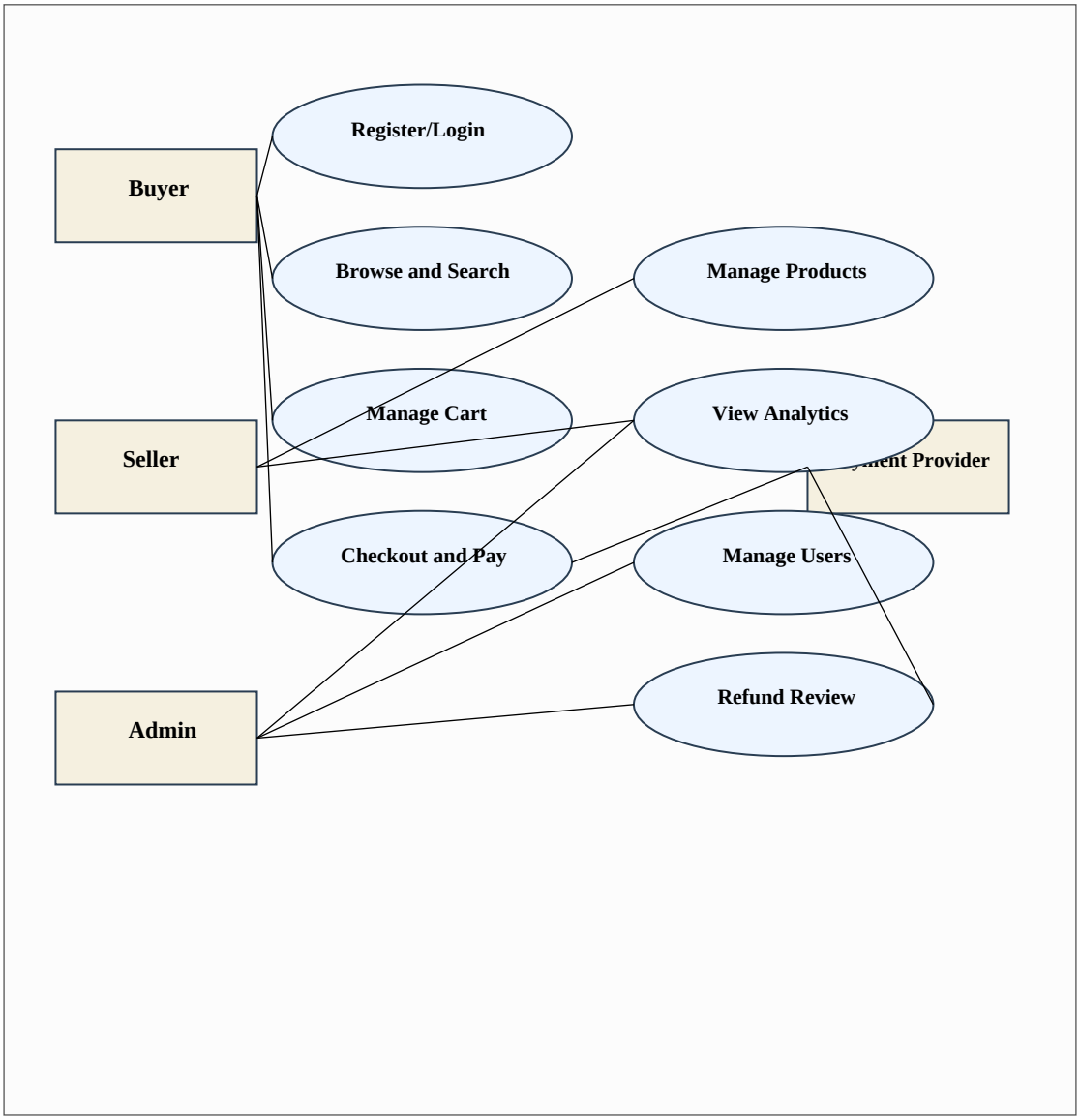


Explanation: The workflow explains how product browsing, cart, checkout, auth, inventory, payment, webhook, order update, and notification are connected.

3.4 Use Case Diagram

The use case diagram shows the main actors and their interactions with the platform. Buyer uses shopping features, seller uses product and analytics features, admin uses platform management features, and payment provider interacts with checkout and refund flows.

Figure 5: Use Case Diagram

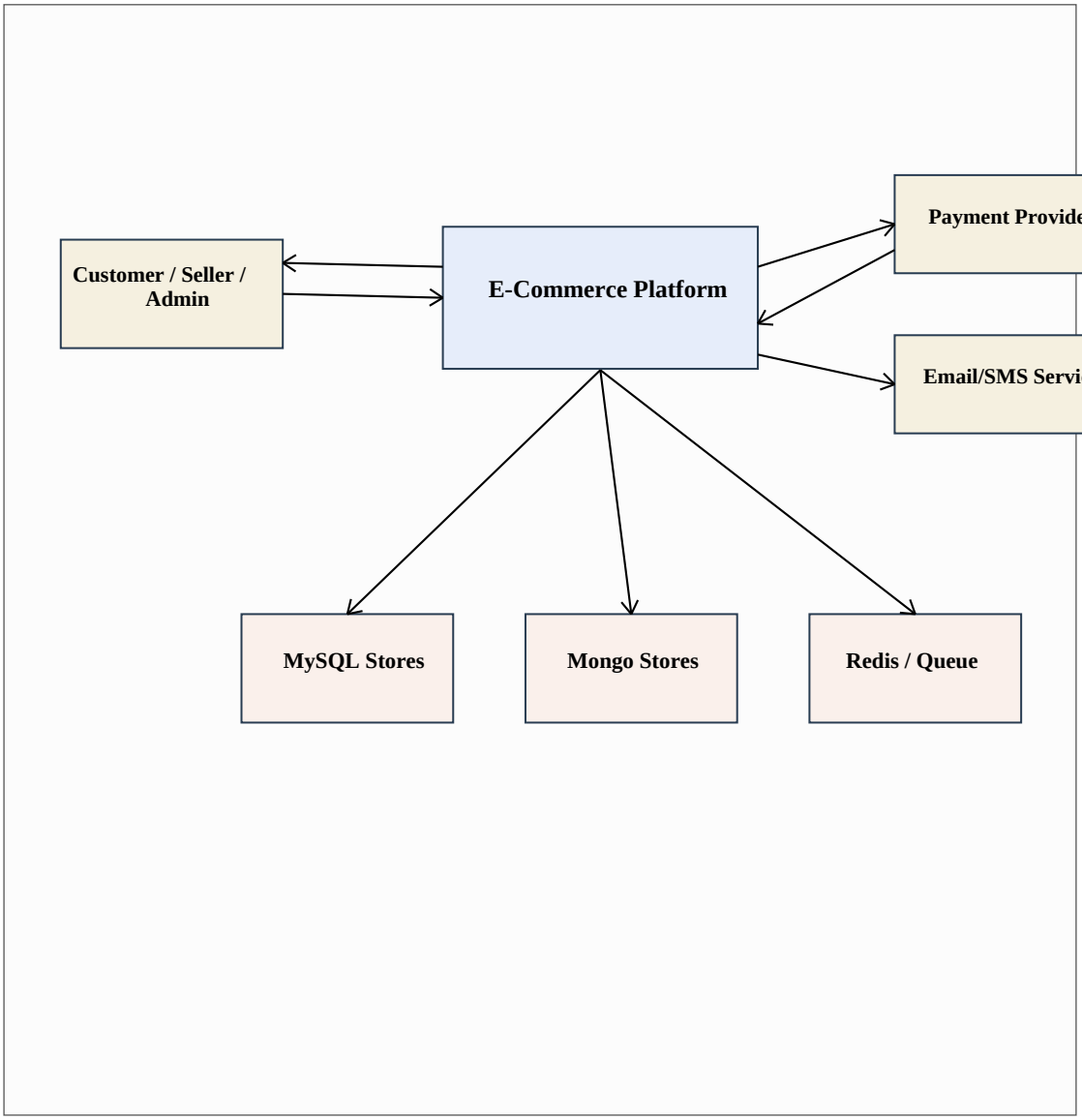


Explanation: The diagram shows buyer, seller, admin, and payment provider as actors. It also shows the main user-facing and admin-facing use cases.

3.5 Data Flow Diagram

Data flow design explains how data moves between users, the platform, external providers, and data stores. In this system, users and dashboards do not directly access databases. They use the API Gateway and services.

Figure 6: Data Flow Diagram - Level 0



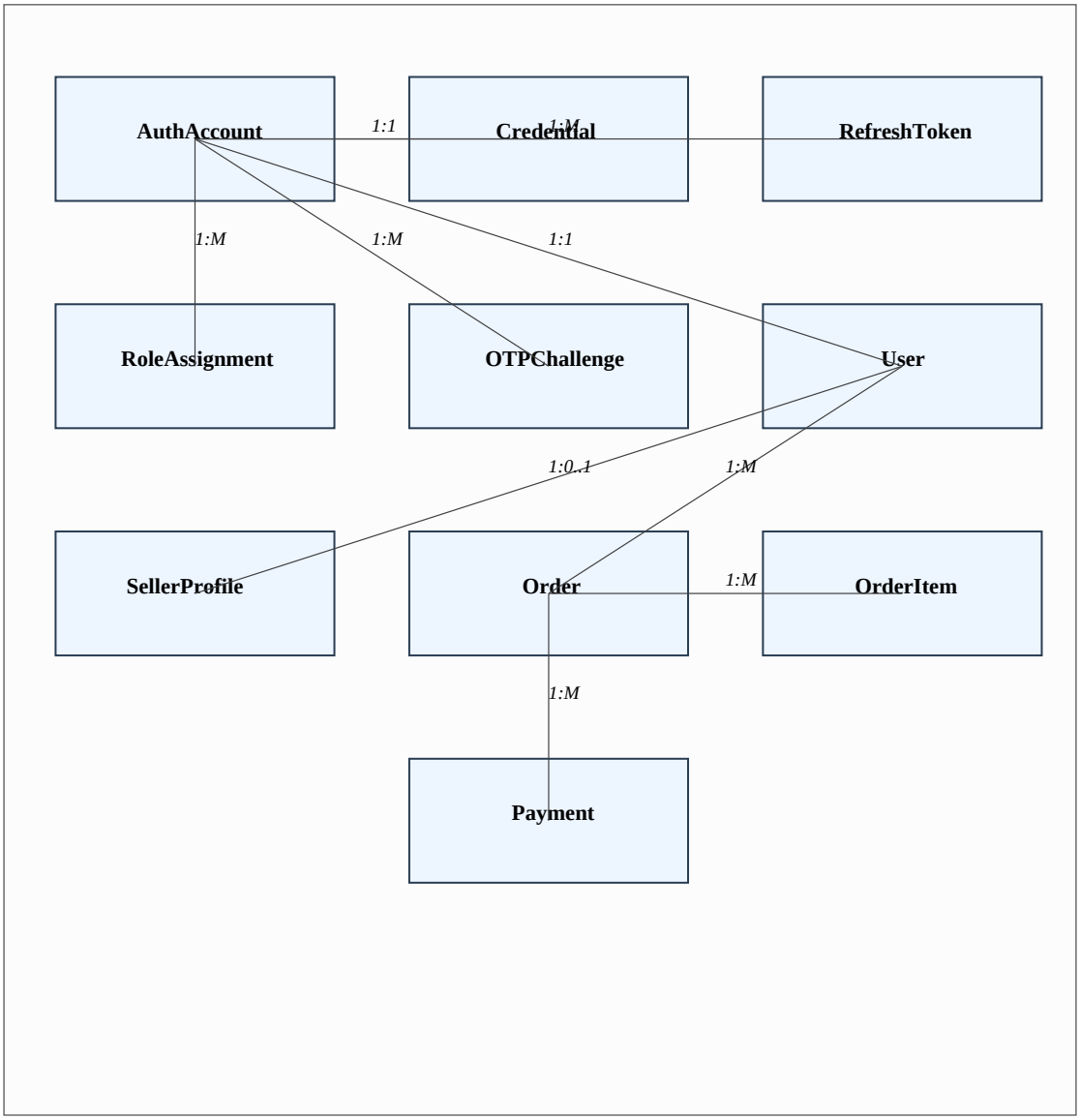
Explanation: The DFD shows external users, the e-commerce platform, payment provider, notification provider, and main data stores. It keeps the view simple for beginner understanding.

3.6 Database Design

The platform uses database-per-service ownership. This means every service owns its own database or collections. No service is allowed to directly use another service's database tables. This rule keeps the system modular and reduces hidden coupling.

Service	Database	Reason
Auth	MySQL + Redis	Credentials and tokens need consistency; Redis supports rate limits and short-lived state.
User	MySQL	Profiles, addresses, and seller KYC are structured relational data.
Product	MongoDB	Product attributes and variants are flexible by category.
Cart	MongoDB + Redis	Cart document is flexible; Redis helps hot cart reads.
Wishlist	MongoDB	Wishlist is document-friendly and read-heavy.
Order	MySQL	Orders need transaction and audit records.
Payment	MySQL	Payment and refund data need strong audit and consistency.
Search	Typesense	Fast text search, filters, facets, and sorting.
Session	MongoDB + Redis	High-volume flexible events and active sessions.
Notification	MongoDB	Templates and provider payloads vary.
Superadmin	MySQL	Permissions, settings, tasks, and audit logs are structured.

Figure 7: Entity Relationship Diagram



Explanation: The ER diagram focuses on important relational entities from authentication, user, order, and payment areas. It shows one-to-one and one-to-many relations.

3.7 Important MySQL Entities

The relational schema includes separate databases such as auth_db, user_db, order_db, payment_db, cms_db, and admin_db. The implemented Auth Service migrations define auth_accounts, credentials, refresh_tokens, otp_challenges, role_assignments, and auth_outbox_events.

Entity	Purpose
auth_accounts	Stores account id, email, phone, verification flags, and account status.
credentials	Stores password hash, algorithm, failed attempts, and lockout time.
refresh_tokens	Stores hashed refresh tokens with session id, expiry, and revoke time.
otp_challenges	Stores OTP challenge id, target, channel, purpose, hash, attempts, and expiry.
role_assignments	Stores account role, scope, assigned by, assigned at, and revoke status.
auth_outbox_events	Stores auth events for async publishing to session/fraud systems.
orders	Stores order id, user id, status, amount fields, address snapshot, and payment id.
order_items	Stores item snapshot, seller id, product id, variant id, quantity, and fulfillment status.
payments	Stores payment status, provider references, amount, and audit details.

3.8 MongoDB Collections

MongoDB is used where records are flexible and document-based. Product documents can contain category-specific attributes and variant arrays. Session events are stored as separate documents because events can be very high in number.

Collection	Service	Purpose
products	Product	Product details, attributes, images, variants, price, stock, rating.
categories	Product	Category tree and display order.
carts	Cart	Active cart items, coupon, totals, expiry.
wishlists	Wishlist	Saved products and last known availability.

Collection	Service	Purpose
user_interactions	Recommendation	Product view and behavior events for recommendation.
sessions	Session	Session id, anonymous id, user id, device, geo, start, last seen.
session_events	Session	Page view, click, search, add to cart, checkout events.
heatmap_points	Session	Aggregated click or scroll positions.
notification_templates	Notification	Email/SMS/push template definitions.
notification_deliveries	Notification	Delivery status, provider id, attempts, and payload.

3.9 API Design

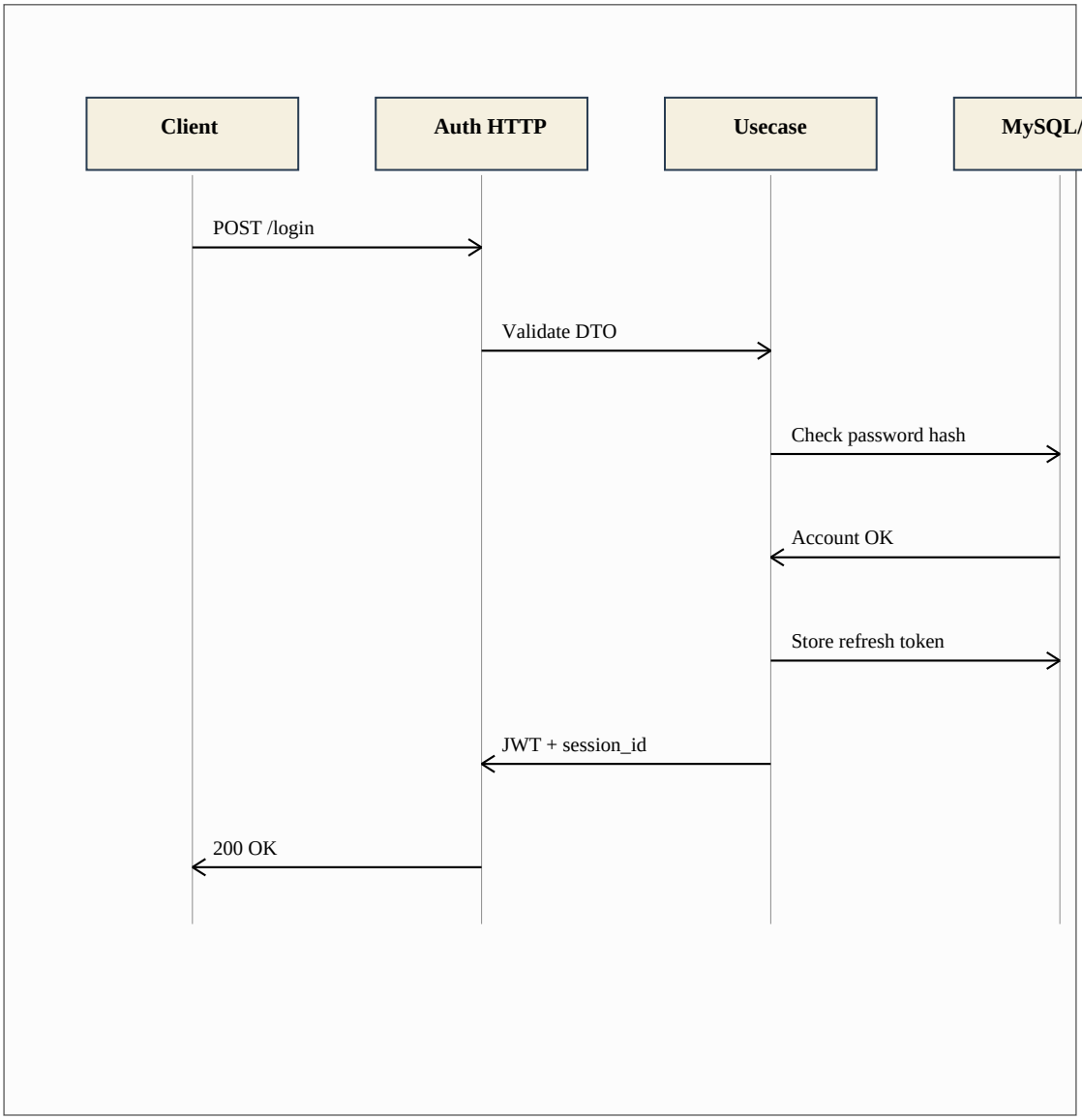
The platform exposes REST APIs through the API Gateway. Internal services can use gRPC when typed and fast service-to-service calls are needed. Public browser APIs use paths like /api/v1/auth/login and /api/v1/analytics/live.

API	Method	Purpose
/api/v1/auth/login	POST	Login user and return access token, refresh token, and session id.
/api/v1/auth/refresh	POST	Rotate refresh token and issue a new access token.
/api/v1/auth/logout	POST	Revoke refresh token and end session.
/api/v1/auth/otp/send	POST	Create OTP challenge and send OTP through notification provider.
/api/v1/auth/otp/verify	POST	Verify OTP for signup, login, or password reset.
/api/v1/search	GET	Search products with query, filters, facets, and sorting.
/api/v1/orders/checkout	POST	Create order from cart and initiate payment.
/api/v1/webhooks/payments/{provider}	POST	Receive and verify payment provider webhook.
/api/v1/analytics/live	GET	Fetch live session metrics for dashboard filters.
/api/v1/analytics/heatmaps	GET	Fetch heatmap data for selected route and segment.

3.10 Security Design

Security is treated as a foundation of the platform. The Auth Service handles password hashing, JWT token issue, refresh token rotation, OTP challenges, role assignment, and logout. Sensitive values such as OTP, refresh tokens, passwords, card data, and raw secrets must not be logged.

Figure 8: Authentication Flow

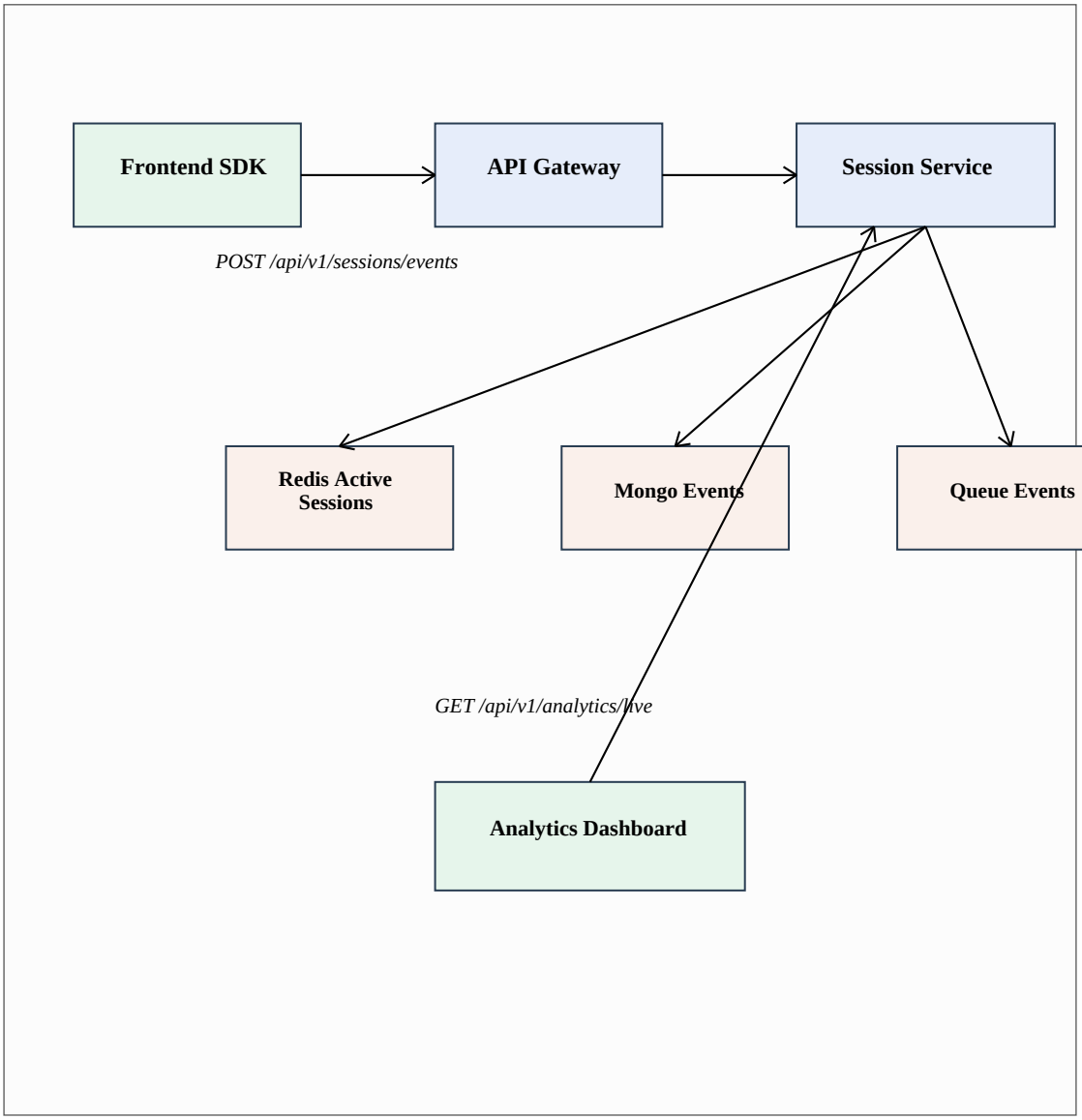


Explanation: The flow shows how login passes through HTTP transport, usecase layer, MySQL/Redis repositories, and returns token/session details to the client.

3.11 Session Analytics Design

The Session Management Service is independent from basic login. It tracks anonymous and logged-in sessions, page views, clicks, searches, product views, add-to-cart events, checkout steps, payment result events, journeys, funnels, heatmaps, and live metrics. Privacy rules say that raw IP, OTP, passwords, card data, and private text fields must not be collected.

Figure 9: Session Analytics Data Flow



Explanation: The diagram shows frontend event collection, gateway forwarding, session service processing, Redis active-session updates, MongoDB event storage, queue publishing, and dashboard metric reading.

3.12 Privacy and Data Retention Design

Session analytics is useful, but it can also become sensitive if personal or behavioral data is shown without control. Therefore, the design includes privacy rules. Raw IP should not be stored unless a strict security reason exists. IP hash, user id, anonymous id, and session id should be masked by default in admin screens. Passwords, OTP, card data, private text fields, and keystrokes should never be collected in analytics events.

Data	Privacy Rule	Retention Direction
Raw session events	Avoid PII and sensitive fields.	TTL based retention, for example 30 to 90 days.
Journey summaries	Use masked user/session ids.	Medium retention, for example 180 to 365 days.
Heatmap points	Store aggregated coordinates, not private text.	Longer retention because it is aggregate data.
Analytics aggregates	Keep identity-free counts and rates.	Long-term storage allowed if no direct identity exists.
IP address	Do not show raw IP. Use hash if needed.	Keep only as long as security policy requires.
Deletion requests	Delete or anonymize linkable session data.	Audit request and result safely.

3.13 Caching and Scalability Design

The platform separates hot data from durable data. Redis is used for values that need fast access, such as active sessions, cart count, OTP retry counters, rate limit buckets, product summary cache, seller settings cache, and platform settings cache. Durable data remains in MySQL or MongoDB.

Data	Cache	Typical TTL
Product summary	Redis or CDN edge where safe	5 to 15 minutes
Category tree	Redis	30 minutes
Cart count	Redis	5 minutes
Search autocomplete	Redis	1 to 5 minutes
Seller settings	Redis	10 minutes
Platform settings	Redis	5 minutes

Data	Cache	Typical TTL
Recommendations	Redis	15 minutes
Active sessions	Redis	Session inactivity window

3.14 Observability Design

Observability helps developers and operations teams understand what is happening inside the platform. The project documents recommend structured JSON logs, Prometheus metrics, distributed tracing through OpenTelemetry, and dashboards or alerts through Grafana.

Observability Area	Examples
Logs	timestamp, level, service, environment, request_id, trace_id, route, error_code, latency_ms.
Metrics	request count, latency p95/p99, error rate, DB latency, Redis latency, queue lag.
Business metrics	signup rate, product views, add to cart rate, checkout started, payment success rate.
Traces	search request, checkout, payment webhook, seller product publish, admin refund review.
Alerts	gateway 5xx, checkout latency, payment webhook failures, queue lag, DB CPU, Redis memory.

3.15 Payment Design

Payment Service abstracts the external payment provider. The final payment status is not decided by the frontend success page. It is decided by the verified provider webhook. This prevents false success states and makes financial records more reliable.

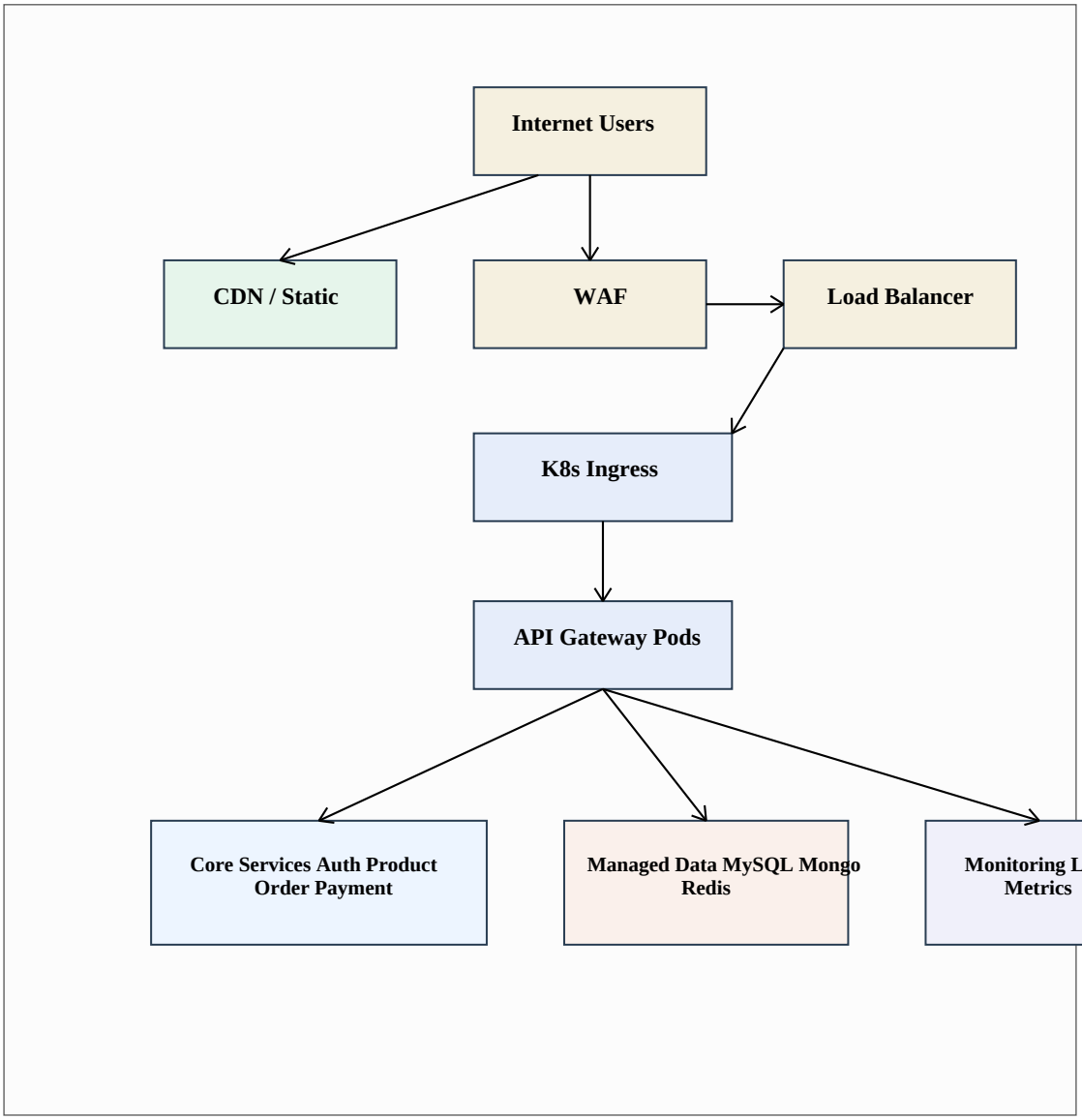
Payment State	Meaning
initiated	Payment intent was created.
requires_action	Provider needs extra user action such as OTP or bank confirmation.
authorized	Provider authorized the payment.
captured	Payment is successful and money is captured.
failed	Payment failed and order can move to payment_failed.
retry_allowed	User can retry payment for the same order.

Payment State	Meaning
partially_refunded	Some amount was refunded.
refunded	Full amount was refunded.

3.16 Deployment Design

The deployment design uses Docker images and Kubernetes manifests. Public traffic reaches the cloud load balancer, WAF, ingress controller, and API Gateway pods. Internal services are exposed through ClusterIP services. Observability tools collect logs, metrics, and traces.

Figure 10: Deployment Architecture



Explanation: The deployment diagram shows the path from internet users to Kubernetes services, managed data stores, and monitoring tools.

Coding & Implementation

4.1 Repository Structure

The repository is organized into documentation, API reference, database design, backend services, frontend apps, and task implementation notes. This structure follows the project documentation and keeps each concern separate.

```
Ecommerce/
  README.md
  Project Work-Assignment 1.pdf
  docs/
  api/master-api.json
  database/draw.sql
  database/mongodb-schema-design.md
  backend/services/auth-service/
  frontend/session-analytics-dashboard/
  TaskImplementation/
  reports/
```

4.2 Backend Auth Service Implementation

The Auth Service is implemented in Go. The service starts from cmd/server/main.go. It loads configuration, connects to MySQL, connects to Redis, initializes password hashing, OTP generator, OTP hasher, notification client, repositories, JWT issuer/verifier, session linker, use cases, HTTP handler, router, and HTTP server.

Layer	Files / Package	Responsibility
cmd/server	main.go	Application bootstrap and dependency wiring.
config	internal/config	Load HTTP, DB, Redis, token, OTP, password, and session-link settings.
domain	internal/domain	Account, credential, token, OTP, role, outbox, and domain errors.
usecase	internal/usecase	Login, token, password, OTP, role, reset password, and signup workflows.
repository	internal/repository	MySQL and Redis repository implementations.
security	internal/security	Password hashing, token issue/verify, OTP generator and hasher.
transport	internal/transport/http	HTTP routes, DTO mapping, middleware, and error responses.

Layer	Files / Package	Responsibility
events	internal/events	Outbox worker and event publishing.
sessionlink	internal/sessionlink	Session-link events and privacy hashing.

4.3 Implemented Auth Routes

The HTTP router registers public auth routes, internal service routes, role management routes, and JWKS route. The health route /healthz returns status ok.

Route	Purpose
/api/v1/auth/login	Login and receive user, token pair, and session id.
/api/v1/auth/refresh	Refresh access token using refresh token rotation.
/api/v1/auth/logout	Logout and revoke refresh token.
/api/v1/auth/otp/send	Create OTP challenge and send OTP.
/api/v1/auth/otp/verify	Verify OTP challenge.
/api/v1/auth/password/forgot	Start password reset OTP challenge.
/internal/v1/auth/credentials	Create credential for an account.
/internal/v1/auth/password/verify	Verify password internally.
/internal/v1/auth/tokens/issue	Issue token pair internally.
/internal/v1/auth/tokens/verify	Verify access token internally.
/internal/v1/auth/roles	Read user roles.
/internal/v1/auth/roles/assign	Assign roles to allowed admins.
/internal/v1/auth/roles/revoke	Revoke roles from allowed admins.
/.well-known/jwks.json	Expose JSON Web Key Set for token verification.

4.4 Password and Token Implementation

The password module supports secure password hashing through a router. The project documents mention Argon2id or bcrypt. The implementation stores password hash and algorithm metadata in credentials table. Failed login attempts and lockout time are also stored.

The token module issues JWT access tokens and stores refresh token hashes. Refresh token rotation is used. This means a refresh request should replace the old refresh token with a new one. If an old token is reused, it can be treated as a fraud or compromise signal.

4.5 OTP Implementation

The OTP flow creates an OTP challenge, stores only the OTP hash, sends OTP through the notification client, and verifies the submitted OTP against the stored hash. The policy includes expiry, maximum attempts, resend cooldown, and rate limits. Redis is used for OTP rate counters.

4.6 RBAC Implementation

Role-Based Access Control is used to control access to sensitive routes. Roles include buyer, seller, admin, finance_admin, catalog_admin, operations_admin, and superadmin. High-risk role mutation routes are protected by authentication and required role middleware.

4.7 Outbox and Session-Link Implementation

The Auth Service includes auth_outbox_events table and an outbox worker. This supports reliable asynchronous event publishing. For example, login, logout, token reuse, or session-link events can be stored first and then published to the Session Service or fraud systems.

4.8 Auth Database Migrations

The Auth Service migrations create and update the authentication database. The first migration creates auth_accounts, credentials, refresh_tokens, otp_challenges, and role_assignments. Later migrations add token claim metadata, OTP account foreign key, hardened role assignment fields, and outbox events.

Migration	**Purpose**
001_create_auth_tables	Creates main auth tables.
002_add_token_claim_metadata	Adds metadata needed for token claims.
003_add_otp_account_fk	Hardens OTP relation with auth account.
004_harden_role_assignments	Improves role assignment safety.

Migration	Purpose
005_create_auth_outbox_events	Creates durable event outbox table.

4.9 Session Analytics Dashboard Implementation

The Session Analytics Dashboard is implemented as a Vite React TypeScript frontend app. It uses TanStack Query for server data, React Router for app routing, Tailwind CSS for styling, lucide-react for icons, date-fns for date helpers, Vitest and Testing Library for tests, and MSW for API mocking.

File / Folder	Purpose
src/main.tsx	Bootstraps the React application.
src/app.tsx	Defines providers and route structure.
src/layout/analytics-layout.tsx	Creates dashboard layout.
src/layout/filters-bar.tsx	Date range controls, segment filters, and refresh button.
src/features/shell/pages/analytics-overview-page.tsx	Main overview page for live metrics.
src/features/shell/components/metric-card.tsx	Single metric display component.
src/features/shell/components/metric-card-grid.tsx	Responsive metric grid.
src/features/shell/components/segment-filter-panel.tsx	Device, channel, source, and user type filters.
src/api/session-api.ts	Maps /api/v1/analytics/live DTO to frontend domain model.
src/services/session-analytics-service.ts	Validates request and calls analytics repository.
src/domain/analytics.ts	Defines dashboard domain types.
src/domain/validation.ts	Validates date range and filter request.

4.10 Dashboard Metrics

The live metrics model includes active users now, active sessions, events per minute, sessions today, conversion rate, average session duration, bounce rate, product view to cart rate, cart to checkout rate, checkout to paid rate, and generated at timestamp.

Metric	Meaning
activeUsersNow	Number of active users currently using the platform.
activeSessions	Number of active session windows.
eventsPerMinute	Recent event ingestion speed.
sessionsToday	Number of sessions created today.
conversionRate	Overall conversion percentage.
averageSessionDurationSeconds	Average length of user session.
bounceRate	Sessions with no deeper interaction.
productViewToCartRate	How many product views become cart additions.
cartToCheckoutRate	How many carts reach checkout.
checkoutToPaidRate	How many checkouts become paid orders.

4.11 API Mapping in Dashboard

The dashboard repository calls GET /api/v1/analytics/live with from, to, device_type, channel, source, and user_type query parameters. The response mapper accepts both snake_case and camelCase field names. This makes the frontend more tolerant during backend API evolution.

```
GET /api/v1/analytics/live
Query:
  from=YYYY-MM-DD
  to=YYYY-MM-DD
  device_type=all|desktop|mobile|tablet
  channel=all|web|mobile_web|app
  source=all|direct|search|paid|social|email
  user_type=all|anonymous|logged_in
```

4.12 Implementation Limitations

The repository currently contains a strong design for many services, but not all services are fully implemented in source code. This report therefore separates documented design from implemented code. The implemented code areas are Auth Service and Session Analytics Dashboard. Product, cart, order, payment, CMS, superadmin, search, recommendation, and notification services are included as planned modules based on the project documentation.

4.13 Configuration Implementation

The Auth Service configuration is loaded during startup. The configuration covers HTTP address and timeouts, database DSN, Redis connection settings, password hashing policy, OTP policy, token issuer and audience, key paths, refresh token TTL, notification endpoint, RBAC settings, and session-link event publishing mode. Validating configuration at startup is important because a wrong secret, missing key, or invalid timeout can make a security service unsafe or unavailable.

Config Area	**Example Values / Purpose**
HTTP	address, read timeout, write timeout, idle timeout.
Database	MySQL DSN for auth_db.
Redis	address, password, DB number, dial/read/write timeout.
Password	hashing algorithm, memory/time cost, max failed attempts, lockout duration.
OTP	length, expiry, attempts, resend cooldown, rate limit pepper.
Token	issuer, audience, key id, private/public key path, access TTL, refresh TTL.
Notification	OTP send endpoint and timeout.
Session link	mode, auth events topic, outbox worker batch size, retry/backoff settings.

4.14 Error Handling and Response Format

The HTTP transport layer converts use case errors into API errors. This keeps internal domain errors separate from public response messages. The handler also limits request body size, checks HTTP method, decodes JSON, normalizes request data, and writes JSON responses.

```
Success response idea:
{
  "data": { ... },
  "request_id": "req_123",
  "error": null
}

Error response idea:
{
  "data": null,
  "request_id": "req_123",
  "error": { "code": "INVALID_REQUEST", "message": "Invalid request" }
}
```

4.15 Coding Standards Followed

The backend uses a clean layered structure. Domain files contain core data and rules. Use cases contain application workflows. Repository files contain database operations. Transport files only map HTTP requests and responses. This makes the code easier to test because use cases can depend on interfaces instead of direct database implementations.

The frontend follows a feature-based structure. Shared types are placed in the domain folder. API mapping is kept inside api/session-api.ts. UI components are separated into metric cards, filter panel, layout, and page files. This keeps the dashboard easy to extend with future pages such as live sessions, journey explorer, funnels, heatmaps, cohorts, reports, and privacy controls.

Testing

Testing is important because the platform handles sensitive actions such as login, OTP, role assignment, checkout, payment, refunds, and admin operations. The testing plan covers backend unit tests, security tests, frontend unit tests, component tests, API mock tests, integration tests, and future end-to-end tests.

5.1 Backend Testing

The Go Auth Service contains several test files for password use case, OTP use case, token use case, role security, auth use case, session-link events, password security, OTP security, token security, HTTP security, and RBAC security. These tests are focused on important security behavior.

Test Area	Purpose
Password tests	Verify password hashing, password checks, reset, lockout, and failed attempts.
OTP tests	Verify OTP generation, hashing, attempts, expiry, and verification behavior.
Token tests	Verify JWT issue, validation, refresh token rotation, and security cases.
Role tests	Verify role assignment, revoke, and permission checks.
HTTP security tests	Verify methods, body limits, error mapping, auth middleware, and protected routes.
Session-link tests	Verify auth event creation and privacy-safe session linking.

5.2 Frontend Testing

The Session Analytics Dashboard includes tests for API mapping, date range helpers, formatting helpers, metric card component, segment filter panel, and analytics overview page. MSW is used for mocking API calls in tests.

Frontend Test	Purpose
session-api.test.ts	Checks API response mapping for live metrics.
date-range.test.ts	Checks date preset and custom date helper behavior.

Frontend Test	Purpose
format.test.ts	Checks number, percent, and duration formatting.
metric-card.test.tsx	Checks metric card display states.
segment-filter-panel.test.tsx	Checks segment filter interactions.
analytics-overview-page.test.tsx	Checks overview page loading, data, error, and empty states.

5.3 Test Cases

Test Case	Input / Action	Expected Result
Login success	Valid identifier and password	User receives access token, refresh token, and session id.
Login failure	Wrong password	System returns auth error and increases failed attempt count.
Refresh token	Valid refresh token	New token pair is issued and old refresh token is rotated.
Refresh reuse	Old refresh token reused	System detects risky reuse and can revoke token family.
OTP send	Valid target and purpose	Challenge id is created and OTP send request is accepted.
OTP verify success	Correct OTP before expiry	Challenge is marked verified.
OTP verify failure	Wrong OTP many times	Attempts are counted and max attempts are enforced.
Role assign	Allowed admin assigns role	Role assignment is stored with audit details.
Analytics filter	Valid date range and filters	Dashboard fetches metrics from live analytics API.
Analytics invalid range	Date range exceeds max days	Dashboard shows validation message and avoids API call.
Payment webhook	Valid provider event id and signature	Payment status is updated idempotently.
Checkout duplicate	Repeated idempotency key	System prevents duplicate order/payment creation.

5.4 Integration Testing Plan

Integration testing should be done after more backend services are implemented. The main integration workflows are signup with OTP, login and refresh token, product search, add to cart, checkout, payment success, payment failure retry, seller product creation, superadmin seller approval, refund review, and analytics event ingestion.

5.5 Non-Functional Testing

Testing Type	What to Check
Performance	Search latency, checkout p95 latency, API Gateway throughput, and session event ingestion rate.
Security	JWT validation, RBAC, OTP attempt limits, rate limits, webhook signatures, and secret redaction.
Reliability	Retry behavior, dead-letter queue, idempotency, graceful shutdown, and health endpoints.
Compatibility	Frontend behavior on desktop, tablet, mobile width, and modern browsers.
Scalability	Horizontal scaling of stateless services and queue consumers.
Usability	Simple dashboard filters, clear errors, readable metrics, and accessible controls.

5.6 Verification Performed for This Report

The report was generated from the local repository documents and code. The PDF generation was verified using a local Python script that writes a valid PDF file and checks that the output file exists with expected page objects. Because Node.js is not usable in this environment, frontend npm tests could not be executed here. Go tests can be executed separately in the backend workspace.

5.7 Suggested Test Commands

The following commands are useful for validating the project in a normal developer setup. Some commands depend on local tools such as Go, Node.js, npm, and running database services.

```
# Backend Auth Service
cd backend/services/auth-service
go test ./...
go vet ./...

# Frontend Session Analytics Dashboard
cd frontend/session-analytics-dashboard
npm run typecheck
npm run test
npm run build

# Report generation
python3 reports/generate_mca_report.py
```

5.8 Acceptance Criteria

Area	Acceptance Criteria
Report	Follows guideline structure, A4 layout, Times font family, 1.5 spacing, page numbering, and required sections.
Auth Service	Login, token refresh, logout, OTP, password reset, JWKS, and RBAC routes are registered.
Database	Auth migrations create required tables and indexes without cross-service database dependency.
Security	Passwords, OTPs, refresh tokens, and secrets are not stored or logged in plain form.
Dashboard	Date range, segment filters, live metric cards, validation, loading, empty, and error states work.
API mapping	Dashboard maps both snake_case and camelCase live metrics responses.
Testing	Important backend and frontend behaviors have focused test files.
Documentation	Design and implemented scope are clearly separated.

Application

The project can be used as the foundation for a modern online shopping platform. It is suitable for systems where buyers, sellers, and admins work at the same time and where future growth is expected.

6.1 Practical Applications

- Online retail marketplace similar to Amazon or Flipkart style platforms.
- Seller dashboard for product, inventory, order, coupon, and revenue management.
- Superadmin panel for platform operations, user management, seller approval, refund review, and audit logs.
- Session analytics dashboard for live metrics, journey analysis, funnels, heatmaps, device reports, and privacy controls.
- Secure authentication service for any platform needing login, OTP, refresh tokens, and RBAC.
- Reference architecture for students learning microservices, database-per-service design, and DevOps.

6.2 Benefits

Benefit	**Explanation**
Scalability	Each service can scale based on its own workload.
Security	Auth, RBAC, OTP, token rotation, rate limits, and audit logs reduce risk.
Maintainability	Clear module boundaries make changes easier.
Performance	Redis cache, Typesense search, and async queues support faster user experience.
Observability	Logs, metrics, traces, and alerts help detect problems early.
Flexibility	MongoDB supports flexible product and event documents.
Reliability	Idempotency and outbox/event patterns reduce duplicate or lost operations.

6.3 Users of the System

User Type	Main Use
Buyer	Search products, manage cart, checkout, view orders, wishlist, and profile.
Seller	Manage products, inventory, orders, coupons, campaigns, staff, and analytics.
Admin	Manage users, sellers, orders, payments, settings, search, sessions, and audit logs.
Developer	Develop services, APIs, frontend modules, tests, and deployments.
Operations Team	Monitor health, logs, metrics, alerts, queue lag, and deployment status.

Conclusion

The project ’Scalable Backend Development for High-Performance E-Commerce Website Platform’ presents a clear design for a scalable and secure e-commerce platform. It follows a microservices architecture, uses database-per-service ownership, includes security controls, and plans for caching, search, message queue, Kubernetes deployment, and observability.

The current implementation proves important parts of the design through a Go Auth Service and a React TypeScript Session Analytics Dashboard. The Auth Service demonstrates practical backend development with password security, OTP, JWT, refresh tokens, RBAC, MySQL, Redis, HTTP routes, and outbox events. The dashboard demonstrates frontend implementation with filters, typed API mapping, metric cards, validation, and tests.

This project helped in understanding real software engineering work such as requirement analysis, modular design, secure coding, database design, API planning, testing, deployment planning, monitoring, and documentation. The project is suitable for future expansion into a complete production-ready e-commerce platform.

Future Enhancements

- Implement remaining services such as User, Product, Cart, Order, Payment, CMS, Search, Recommendation, Notification, and Superadmin.
- Add gRPC/protobuf contracts and generated clients.
- Complete API Gateway implementation with routing, validation, and rate limiting.
- Add full Docker Compose stack and Kubernetes manifests.
- Add integration tests and end-to-end tests for checkout and payment flows.
- Add charts, journey explorer, funnels, heatmaps, cohorts, and privacy controls in the analytics dashboard.
- Add CI/CD pipeline, image scanning, deployment automation, and monitoring dashboards.

- Add production-grade secret management and backup/recovery plan.

Bibliography (APA Style)

- Chandigarh University, Centre for Distance & Online Education. (2025). Guidelines for Major Project (23ONMCR-753).
- Ecommerce project repository. (2026). README.md, architecture documents, API reference, database design, Auth Service source code, and Session Analytics Dashboard source code.
- The Go Authors. (2026). The Go programming language documentation. https://go.dev/doc/
- React Team. (2026). React documentation. https://react.dev/
- TypeScript Team. (2026). TypeScript documentation. https://www.typescriptlang.org/docs/
- MySQL. (2026). MySQL 8.0 reference manual. https://dev.mysql.com/doc/
- MongoDB. (2026). MongoDB manual. https://www.mongodb.com/docs/
- Redis. (2026). Redis documentation. https://redis.io/docs/
- Kubernetes Authors. (2026). Kubernetes documentation. https://kubernetes.io/docs/
- OWASP Foundation. (2026). OWASP application security guidance. https://owasp.org/
- OpenTelemetry. (2026). OpenTelemetry documentation. https://opentelemetry.io/docs/
- Prometheus Authors. (2026). Prometheus documentation. https://prometheus.io/docs/
- Grafana Labs. (2026). Grafana and Loki documentation. https://grafana.com/docs/
- Typesense. (2026). Typesense documentation. https://typesense.org/docs/
- TanStack. (2026). TanStack Query documentation. https://tanstack.com/query/